목차

바이브코딩 + OpenClaw 특강

타임라인

Part 1. 왜 바이브코딩인가 (30분)

Part 2. 환경 세팅 (30분)

현재 내가 어디 있는지 보기

폴더 안에 뭐가 있는지 보기

폴더 이동

한 칸 위로

새 폴더 만들기

파일 내용 보기

1. Claude Code 설치 (터미널에 복붙)

2. 로그인

3. 프로젝트 폴더 만들고 들어가기

4. Claude Code 시작

Part 3. 첫 프로젝트: 텔레그램 봇 (60분)

환경변수 설정 후 실행

간단한 방법 (PM2)

상태 확인

Part 4. 24시간 일하는 AI 비서: OpenClaw (40분)

SOUL.md

AGENTS.md

1. 설치

2. 초기 설정 (대화형)

3. 텔레그램 연동

→ BotFather에서 봇 만들기

→ 토큰을 config에 등록

4. 실행

Part 5. 실전 활용 사례 + Q&A (30분)

부록: 비용 정리

부록: 자주 묻는 질문

강사 메모

바이브코딩 + OpenClaw 특강

대상: 코딩 경험 없는 비개발자 (크립토/스타트업/VC 업계) **시간**: 3~4시간 (휴식 포함) **목표**: 수강 후 혼자서 간단한 봇/대시보드를 만들 수 있는 상태

타임라인

시간	파트	내용
0:00~0:30	**1. 왜 바이브코딩인가**	개념, 가능한 것과 불가능한 것, 라이브 데모
0:30~1:00	**2. 환경 세팅**	터미널, Claude Code, 첫 실행
1:00~1:10	휴식	
1:10~2:10	**3. 첫 프로젝트: 텔레그램 봇**	실습. BotFather → 코드 → 실행
2:10~2:20	휴식	
2:20~3:00	**4. 24시간 일하는 AI 비서: OpenClaw**	개념, 세팅, SOUL/AGENTS, 스킬
3:00~3:30	**5. 실전 활용 사례 + Q&A**	Mason의 실제 셋업, 응용 아이디어

Part 1. 왜 바이브코딩인가 (30분)

바이브코딩이 뭔가

"코드를 쓰는 게 아니라, 만들고 싶은 걸 말로 설명하면 AI가 코드를 짜주는 것."

전통적 개발:

```
문제 정의 → 설계 → 코딩 → 디버깅 → 배포
                ↑ 여기서 몇 년 걸림
```

바이브코딩:

```
문제 정의 → AI한테 설명 → AI가 코딩 → 같이 수정 → 배포
                ↑ 여기가 핵심 스킬
```

핵심 전환: "코드를 읽고 쓰는 능력" → "무엇을 만들어야 하는지 명확하게 설명하는 능력"

할 수 있는 것 vs 기대 조절

바이브코딩으로 하루만에 가능한 것: - 텔레그램/디스코드 봇 - 데이터 수집 스크립트 (트위터, 웹사이트 크롤링) - 간단한 대시보드/웹페이지 - API 연동 자동화 (알림, 포워딩) - 엑셀/CSV 데이터 분석 도구

솔직히 말하면 어려운 것: - 대규모 서비스 (카카오톡 같은 앱) - 고성능 실시간 시스템 - 모바일 앱 (가능하지만 마찰이 큼) - "AI가 알아서 완벽하게" — 감독 없이 돌리면 100% 문제 생김

가장 큰 오해: "코딩을 몰라도 된다" ← **반만 맞는 말** - 코딩 "문법"은 몰라도 됨 - 코딩 "개념"은 알수록 유리함 (파일, 폴더, 서버, API, 데이터베이스가 뭔지) - 비유: 건축을 몰라도 집은 지을 수 있지만, "화장실 배관이 뭔지"는 알아야 AI한테 제대로 설명할 수 있음

🔴 라이브 데모 (5분)

[강사 시연] Claude Code로 간단한 것 하나를 실시간으로 만들어 보여주기. - 예: "현재 비트코인 가격을 가져와서 보여주는 웹페이지 만들어줘" - 30초 안에 동작하는 걸 보여줌 → 임팩트

> **강사 노트**: 첫 데모는 "와 진짜 되네"를 보여주는 게 목적. 복잡한 건 나중에.

Part 2. 환경 세팅 (30분)

필요한 것

항목	비용	용도
컴퓨터 (Mac 권장, Windows 가능)	-	작업 환경
VS Code	무료	코드 편집기 (AI가 만든 코드를 볼 수 있는 창)

항목	비용	용도
Claude Pro 구독	$20/월 (~₩28,000)	AI 코딩 도구 사용권
Claude Code (CLI)	무료 (Pro 구독에 포함)	터미널에서 AI와 대화하며 코딩
(선택) **원격 서버**	~₩30,000/월	무거운 작업용 (나중에)

> **왜 Mac?**: 개발 도구들이 Mac/Linux에 최적화되어 있음. Windows도 되지만 마찰이 더 있음. Windows 쓰는 분은 WSL2 (Windows 안의 Linux) 세팅이 필요할 수 있음.

터미널 — 겁먹지 마세요

터미널 = 텍스트로 컴퓨터한테 명령하는 창. 그게 다임.

딱 이것만 알면 됩니다:

```
# 현재 내가 어디 있는지 보기
pwd

# 폴더 안에 뭐가 있는지 보기
ls

# 폴더 이동
cd 폴더이름

# 한 칸 위로
cd ..

# 새 폴더 만들기
mkdir 프로젝트이름

# 파일 내용 보기
cat 파일이름
```

> **강사 노트**: 여기서 수강생이 직접 터미널 열고 위 6개 명령어를 따라 치게 하기. 5분이면 충분.

Claude Code 설치 + 첫 실행

```
# 1. Claude Code 설치 (터미널에 복붙)
npm install -g @anthropic-ai/claude-code

# 2. 로그인
claude login

# 3. 프로젝트 폴더 만들고 들어가기
mkdir my-first-project
cd my-first-project

# 4. Claude Code 시작
claude
```

여기서 Claude가 대화창을 열어줌. 이제부터 **한국어로 말하면 됩니다.**

🔵 실습: 첫 대화

`You: 현재 날짜와 시간을 보여주는 웹페이지를 만들어줘. 한국어로, 예쁘게.`

→ Claude가 HTML 파일을 만들어줌 → 브라우저에서 열기 → "내가 만든" 첫 웹페이지

> **강사 노트**: 이 순간이 중요. "내가 코드를 짠 건 아닌데, 내가 시킨 대로 동작하는 게 나왔다"는 경험.

Part 3. 첫 프로젝트: 텔레그램 봇 (60분)

왜 텔레그램 봇?

- 크립토 업계에서 가장 실용적
- 결과물이 바로 내 손안에 (모바일에서 확인)

- 구조가 단순해서 입문용으로 최적

Step 1. BotFather로 봇 생성 (5분)

1. 텔레그램에서 `@BotFather` 검색
2. `/newbot` 입력
3. 봇 이름/유저네임 설정
4. **API 토큰** 받기 (이걸 잘 저장)

`예: 7123456789:AAF8Gk3x...`

> **강사 노트**: 이 토큰은 "봇의 비밀번호"와 같음. 남한테 공유하면 안 된다고 강조.

Step 2. Claude Code로 봇 만들기 (20분)

```
mkdir my-telegram-bot
cd my-telegram-bot
claude
```

Claude에게:

```
텔레그램 봇을 만들어줘.

기능:
1. /start 하면 "안녕하세요! 저는 [이름]의 봇입니다" 라고 인사
2. /btc 하면 현재 비트코인 가격을 KRW로 보여줘
3. /eth 하면 현재 이더리움 가격을 KRW로 보여줘

기술 스택: Node.js, node-telegram-bot-api
가격 데이터: CoinGecko 무료 API 사용

BOT_TOKEN은 환경변수로 관리해줘.
```

→ Claude가 코드를 만듦 → 설치 → 실행

```
# 환경변수 설정 후 실행
BOT_TOKEN=7123456789:AAF8Gk3x... node bot.js
```

→ 텔레그램에서 내 봇에게 `/btc` 보내기 → 가격 나옴 → 🎉

Step 3. 기능 추가해보기 (15분)

Claude에게 추가 요청 (수강생이 직접):

```
이 봇에 기능 추가해줘:
- /fear 하면 Fear & Greed Index를 보여줘
- 매일 아침 9시에 자동으로 BTC 가격을 보내줘 (내 채팅방에)
```

> **강사 노트**: 여기서 핵심은 "추가 기능을 말로 요청하면 된다"는 체험. 에러가 나면? → 에러 메시지를 그대로 Claude에게 복붙하면 고쳐줌.

Step 4. 24시간 돌리기 (10분)

"터미널 닫으면 봇이 꺼지는데?" → 백그라운드 실행

```
# 간단한 방법 (PM2)
npm install -g pm2
pm2 start bot.js
pm2 save

# 상태 확인
pm2 status
```

> **강사 노트**: 서버 없이 맥미니에서 돌려도 됨. 나중에 트래픽 많아지면 서버 분리.

에러가 났을 때 대처법

바이브코딩의 진짜 스킬 = 에러 핸들링

1. 에러 메시지를 **그대로** 복사
2. Claude에게 붙여넣기
3. "이 에러 고쳐줘"
4. 끝

안 되는 경우 (10번 중 1~2번): - Claude에게 더 구체적으로 설명 (뭘 하다가, 어떤 상황에서) - "다른 방법으로 해줘" - 최후의 수단: 처음부터 다시 (바이브코딩은 "다시 만들기"가 쌈)

Part 4. 24시간 일하는 AI 비서: OpenClaw (40분)

Claude Code vs OpenClaw — 뭐가 다른가

	Claude Code	OpenClaw
비유	콜센터 (전화할 때만 응답)	상주 비서 (항상 대기)
작동 방식	내가 터미널 열고 대화	텔레그램/슬랙으로 아무때나 말 걸기
실행 환경	내 노트북에서	서버에서 24시간
특기	코딩	코딩 + 자동화 + 리서치 + 모니터링
상태 유지	대화 닫으면 리셋	기억이 쌓임 (MEMORY.md)

Claude Code = 코딩할 때 쓰는 도구 **OpenClaw** = Claude Code를 포함해서, 내 모든 일을 대신해주는 에이전트 프레임워크

OpenClaw의 핵심 개념

1. 채널 연동 - 텔레그램, 디스코드, 슬랙, 왓츠앱 등 - 카톡 보내듯이 AI한테 일 시키기 - "야 비트코인 지금 얼마야?" → 텔레그램으로 즉시 답변

2. 스킬 (Skills) - 봇이 할 수 있는 능력을 모듈로 관리 - 예: 날씨, 주식, 이메일, 캘린더, GitHub, 웹검색... - 남이 만든 스킬 설치 가능 (clawhub.com)

3. SOUL.md — 봇의 성격

```
# SOUL.md
- 말투: 반말, 친구처럼
- 성격: 직설적, 유머 있게
- 판단: 내 편에서 최적화
```

→ 같은 OpenClaw라도 SOUL.md에 따라 완전히 다른 봇이 됨

4. AGENTS.md — 봇의 업무 규칙

```
# AGENTS.md
- 매일 아침 9시에 암호화폐 시장 요약 보내기
- 에러나면 즉시 알리기
- 허락 묻지 말고 실행
```

5. MEMORY.md — 봇의 기억 - 대화 내용이 누적됨 - "저번에 말한 그 프로젝트" → 기억하고 있음 - 사람 이름, 선호도, 결정사항 등 장기 기억

🔴 라이브 데모: Mason의 실제 OpenClaw

[강사 시연] 텔레그램에서 실시간으로:

```
Mason: 지금 BTC 가격 알려줘
Nova: 비트코인 현재 $97,234 (~₩141,523,000)입니다. 24h -2.3%.

Mason: 오늘 일정 뭐 있어?
Nova: (Google Calendar 연동) 14:00 팀미팅, 16:00 투자자 콜...

Mason: 이 웹사이트 요약해줘 [URL]
Nova: (요약 결과)
```

> **강사 노트**: "이게 다 텔레그램 안에서 된다"는 걸 보여주는 게 핵심.

OpenClaw 설치 (개요만)

```
# 1. 설치
npm install -g openclaw

# 2. 초기 설정 (대화형)
openclaw init

# 3. 텔레그램 연동
# → BotFather에서 봇 만들기
```

`# → 토큰을 config에 등록`

`# 4. 실행`
`openclaw gateway start`

> **강사 노트**: 특강에서 OpenClaw 전체 세팅까지 하기엔 시간이 부족할 수 있음. "이런 게 가능하다"를 보여주고, 세팅은 별도 세션(1:1 또는 후속 강의)으로. 또는 사전에 세팅된 환경을 제공해서 바로 체험하게 하는 방법도 있음.

Part 5. 실전 활용 사례 + Q&A (30분)

Mason의 실제 활용 사례

1. 일일 크립토 브리핑 - 매일 아침 자동 생성 → 텔레그램 발송 - 소스: X, GitHub, Reddit, Hacker News - "제가 매일 아침 30분 쓰던 일을 봇이 3분 만에 해줌"

2. 호텔 조사 자동화 - 여행 계획 중 "대련 2BR 호텔 100개 조사해줘" - 봇이 Booking.com + Trip.com 535개 검색 → 62개 필터 → 36개 shortlist - API 인터셉션까지 자동으로 해서 45초 만에 전수조사

3. 투자 리서치 - 업종별 분석 → 40개 챕터 → 224페이지 PDF 자동 생성 - 사람이 하면 2주, 봇이 하면 하루

4. 코드 작성/배포 - "이 기능 추가해줘" → 원격 서버에서 코딩 → 배포 → 완료 보고 - 리뷰어 겸 개발자 겸 DevOps

바이브코딩 잘하는 법 — 팁

1. "기획서"를 잘 쓰면 결과가 좋다

❌ "텔레그램 봇 만들어줘"
✅ "텔레그램 봇 만들어줘. 기능: `/price`로 BTC 가격 조회,
`CoinGecko API` 사용, `Node.js`, 1분마다 캐시 갱신"

2. 한 번에 다 시키지 말고, 점진적으로

`1차: 기본 뼈대 → 확인`
`2차: 기능 A 추가 → 확인`
`3차: 기능 B 추가 → 확인`

→ 한 번에 10개 기능 시키면 3개는 이상하게 나옴

3. 에러는 친구 - 에러 = AI가 고칠 수 있는 구체적인 정보 - 에러 없이 "그냥 안 돼요"가 제일 어려움 - 에러 메시지 복붙하면 90%는 자동 해결

4. 코드를 "읽을" 필요는 없지만, "구조"는 이해하면 좋다 - 파일이 뭘 하는지 대략 파악 - 설정 파일은 어디에 있는지 - "이 파일 수정해줘"라고 지정할 수 있으면 속도가 2배

5. Git = 세이브 포인트

```
git init            # 세이브 시스템 시작
git add .           # 현재 상태 찍기
git commit -m "여기까지 잘 됨"  # 세이브
```

→ 뭔가 망가지면 "저장한 시점으로 돌아가기" 가능

다음 단계 가이드

레벨	할 수 있는 것	걸리는 시간
Lv.1 오늘	간단한 봇, 스크립트	오늘 이후 바로
Lv.2 1주 후	웹 대시보드, 크롤러	1주 연습
Lv.3 1달 후	API 서버, 데이터 파이프라인	꾸준히 만들면
Lv.4 3달 후	OpenClaw 에이전트 풀 커스텀	인프라 이해 필요
Lv.5 6달 후	프로덕션 서비스 운영	운영 경험 축적

추천 리소스

- **Claude Code 공식 문서**: https://docs.anthropic.com/en/docs/claude-code
- **OpenClaw 공식 문서**: https://docs.openclaw.ai
- **OpenClaw 스킬 마켓**: https://clawhub.com

- **OpenClaw 커뮤니티**: https://discord.com/invite/clawd

부록: 비용 정리

항목	월 비용	필수 여부
Claude Pro	~₩28,000 ($20)	✅ 필수
Mac Mini (M4, 16GB)	~₩80만 (일시불)	✅ 권장 (Windows도 가능)
VS Code	무료	✅ 필수
원격 서버 (Hetzner 등)	~₩30,000	❌ 나중에
도메인	~₩15,000/년	❌ 선택
월 유지비 (최소)	**~₩28,000**	

> Claude Pro 하나면 Claude Code + API 사용량이 상당히 포함됨. 하루에 10시간 코딩해도 Pro로 보통 충분. 부족하면 Max ($100/월)로 업그레이드.

부록: 자주 묻는 질문

Q: 영어를 잘해야 하나요? A: 아뇨. Claude Code는 한국어 잘 알아들어요. 다만 에러 메시지가 영어로 나올 때가 있는데, 그것도 Claude에게 "이거 뭔 뜻이야?" 하면 됩니다.

Q: 수학/알고리즘을 알아야 하나요? A: 일반적인 바이브코딩에서는 전혀요. AI가 알고리즘을 짜는 거니까. "정렬해줘", "필터링해줘" 이 정도면 충분합니다.

Q: 만든 걸 다른 사람한테 보여줄 수 있나요? A: 네. 웹페이지는 Vercel(무료)에 배포, 봇은 서버에서 돌리면 됩니다. 전부 Claude에게 "배포해줘"라고 하면 도와줍니다.

Q: AI가 만든 코드를 믿을 수 있나요? A: 90%는 맞지만 10%는 확인이 필요해요. 그래서 "작은 단위로 만들고 확인"하는 습관이 중요합니다. 돈이 오가는 기능(결제, 전송)은 반드시 테스트.

Q: Claude Code vs Cursor vs Copilot — 뭘 써야 하나요? A: - **Claude Code**: 터미널에서 대화. 자유도 최고, 학습 곡선 약간 있음. - **Cursor**: VS Code에 AI 내장. 비개발자가 시작하기 가장 쉬움. $20/월. - **GitHub Copilot**: 코드 자동완성 위주. 개발자가 쓰기 좋음. 비개발자한테는 비추. → 오늘 강의는 Claude Code 기준이지만, Cursor도 좋은 선택지.

Q: OpenClaw 말고 다른 에이전트 프레임워크는? A: 여러 가지 있어요 (LangChain, AutoGPT, CrewAI 등). OpenClaw의 차별점은 "설치하면 바로 쓸 수 있는 실용성" + "텔레그램/슬랙 연동" + "스킬 마켓". 다른 건 개발자용이 대부분.

강사 메모

강의 전 준비물

- [ ] Claude Code 라이브 데모용 프로젝트 폴더 (깨끗한 상태)
- [ ] BotFather로 데모용 봇 미리 생성 (토큰 준비)
- [ ] OpenClaw 데모 환경 (텔레그램 연동 상태)
- [ ] 수강생 사전 안내: VS Code 설치, Claude Pro 가입, 텔레그램 설치

수강생 유형별 대응

- **"터미널이 무서워요"**: 정상. 6개 명령어만 연습시키면 됨. GUI 도구 없음을 미리 알림.
- **"에러 나서 안 돼요"**: 에러 메시지 복붙하게 하기. 90%는 Claude가 해결.
- **"더 복잡한 거 하고 싶어요"**: 좋은 신호. 후속 세션/1:1 세팅 제안.
- **"이걸로 돈을 벌 수 있나요?"**: 도구는 도구. 가치를 만드는 건 기획력.

강의 톤

- "여러분도 할 수 있어요!" 톤 ❌ → 과대광고 느낌
- "이렇게 하면 됩니다" 톤 ✅ → 담백하게 보여주기
- 실패 사례도 포함 (AI가 삽질하는 것도 보여주면 오히려 신뢰↑)